

Módulo 1

Fundamentos de programación con .NET Core (C#)



Tipos de datos básicos

Curso de Testing

Index

Estructura básica: “Hola Mundo”	3
Comentarios.....	4
Variables.....	5
Tipos de datos básicos.....	5
Declaración de variables.....	5
Operaciones numéricas.....	6
Operaciones con caracteres.....	7
Concatenación de cadenas.....	7
Operaciones lógicas y de comparación.....	8
Tipos anónimos y dinámicos.....	9
Conversión de tipos de datos.....	9
Escribir y leer de consola.....	12
Caracteres de salto de línea y tabulado.....	12
Formatear la salida de texto.....	13
Leer de la consola.....	14

Estructura básica: “Hola Mundo”

Antes de empezar, vamos a analizar el código que nos provee .NET Core inicialmente en el archivo **Program.cs** cuando creamos un proyecto. Algunos de estos conceptos no se van a entender completamente hasta que no avancemos un poco más en el aprendizaje del lenguaje y sus estructuras.

```
using System;

namespace Mi_Proyecto
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine(";Hola Mundo!");
        }
    }
}
```

La primera instrucción (**using System**) indica que a partir de ese momento podremos utilizar las funcionalidades del “espacio de nombres” System sin necesidad de poner dicho prefijo delante. Por ejemplo, la instrucción más abajo “Console.WriteLine” (que pertenece al espacio de nombres de System) se debería escribir como “System.Console.WriteLine” si no usamos esta instrucción.

Seguidamente vemos la instrucción “**namespace Mi_Proyecto**”. En este caso estamos definiendo un espacio de nombres que se llama como nuestro proyecto. Como veremos en el futuro, esto es útil si queremos crear clases u otras estructuras cuyo nombre pueda existir en el lenguaje C# o en alguna librería (también veremos este concepto).

A continuación (de la instrucción namespace), se abre un **bloque**. Los bloques van agrupados entre llaves, una de apertura ‘{’, y una de cierre ‘}’. Además, las instrucciones dentro del bloque suelen tener una sangría superior (una serie de espacios o tabulador a la izquierda) para ver mejor donde empieza y acaba.

```
namespace Mi_Proyecto
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine(";Hola Mundo!");
        }
    }
}
```

El diagrama muestra el código anterior con tres bloques rectangulares superpuestos para visualizar la estructura de los scopes. El 'BLOQUE 1 - Mi_Proyecto' es un rectángulo rojo que cubre todo el código dentro de las llaves del namespace. El 'BLOQUE 2 - Program' es un rectángulo verde que cubre el código dentro de las llaves de la clase Program. El 'BLOQUE 3 - Main' es un rectángulo azul que cubre el código dentro de las llaves del método Main. Las etiquetas de los bloques están colocadas a la derecha de cada uno.

La instrucción **class Program** define una clase llamada Program. Las clases son las estructuras principales en lenguajes de programación orientados a objetos como C#. El punto de partida de nuestra aplicación siempre tiene que ser una clase. En este caso sería la clase principal del programa.

La instrucción **static void Main(string[] args)** indica que estamos creando un método. Los métodos son los componentes principales de las clases y se encargan de

realizar tareas. Toda aplicación suele tener un método llamado **Main**, que será lo primero que se ejecute al iniciar el programa. Ya aprenderemos como crear los métodos, lo que significan los modificadores **static**, **void**, etc.

Por último, la instrucción **Console.WriteLine("¡Hola Mundo")**, que es la única acción que por ahora hace nuestro programa, imprime un mensaje por la consola. En este caso imprimirá "¡Hola Mundo!". Por tradición, el primer ejemplo que se muestra al aprender cualquier lenguaje de programación, es como imprimir este mensaje para el usuario.

Como se puede observar, todas las instrucciones se finalizan con punto y coma ';', a excepción de aquellas que definen un bloque (que se delimita entre llaves {}).

Comentarios

A lo largo del código se pueden insertar comentarios. Un comentario contiene texto que será ignorado por el compilador y por tanto no estará en el ejecutable final. Sin embargo son útiles para indicar a otros programadores (o para recordar tiempo después) cómo funciona nuestro código. Es decir, para documentarlo.

Por ahora vamos a ver los comentarios más básicos.

- Cuando escribamos 2 barras seguidas '//', todo el texto hasta el final de la línea actual será considerado comentario, y por tanto, ignorado.

```
// Soy un comentario de una línea
```

- Los comentarios de varias líneas empiezan por '/*' y acaban cuando escribimos '*/'.

```
/* Comentario de varias líneas.  
Puedo escribir todo lo que quiera  
hasta el final del comentario */
```

Variables

Un programa no es solo un conjunto de instrucciones estáticas con valores prefijados que imprimimos por consola. Hay datos, como el nombre del usuario, la fecha actual, etc. que son **variables**. Es decir, que durante la ejecución del programa pueden almacenar valores diferentes.

Una variable se compone de 3 partes. El tipo de dato que almacena, el nombre o identificador de la variable y su valor → **tipo nombre = valor**. Se pueden entender el concepto de variable como un valor que está asociado a un nombre. Como este valor no es fijo y normalmente no sabremos cual es, accederemos a él para leerlo o modificarlo a través del nombre de la variable correspondiente.

Tipos de datos básicos

Una variable sólo puede almacenar datos de un determinado tipo. El tipo de datos de una variable se define la primera vez que esta es creada. A partir de ahí no se especifica el tipo cuando leamos o modifiquemos su valor. El tipo tampoco se puede cambiar una vez definido.

Los tipos de datos básicos que soporta C# más usados son:

- Números enteros → **byte** (entero de 8 bits), **short** (entero de 16 bits), **int** (entero de 32 bits), **long** (entero de 64 bits).
- Números decimales → **float** (32 bits), **double** (64 bits), **decimal** (alta precisión → 128 bits).
- Valores lógicos (verdadero o falso) → **bool**. Este tipo de datos puede almacenar un valor que puede ser **true** o **false**.
- Caracteres → **char** (16 bits). Almacenan un carácter en formato Unicode. El carácter a almacenar se indica entre comillas simples → 'a'.
- Cadenas → **string**. No es un tipo básico pero lo vamos a utilizar desde un principio. Almacena una cadena de caracteres que se representa entre comillas dobles → "Hola".

Otros tipos de datos más complejos que veremos más adelante son objetos, arrays, structs, ...

Declaración de variables

Una variable se declara poniendo el tipo delante y a continuación el nombre o identificador de la variable seguido de punto y coma. Hay una serie de reglas para el nombre de una variable.

- Debe empezar por una letra o el símbolo '_'.
- Debe ir seguido de letras, números o guiones.

- No se pueden usar como nombres de variables palabras reservadas del lenguaje. [Lista de palabras reservadas](#).
- Cuando un nombre está compuesto por varias palabras, en C# (y muchos otros lenguajes) se suele usar el estilo [camelCase](#) para unirlos. Empezando siempre por minúscula la primera palabra, el resto irán pegadas y con la primera letra en mayúscula. Ejemplo: **nombreCompletoJugador**.

Aunque se pueden utilizar caracteres específicos de cada idioma como pueden ser los acentos o la letra ñ, tradicionalmente los nombres se suelen limitar al alfabeto inglés.

```
int numero;
double numDecimal;
bool condicional;
char letra;
string cadena;
```

Cuando declaramos varias variables del mismo tipo, los nombres de las mismas se pueden concatenar detrás del tipo de datos separadas por coma.

```
int n1, n2, n3;
```

Una vez declarada la variable, ya se le puede asignar un valor y acceder al mismo en cualquier momento.

```
n1 = 3;
n2 = 4;
n3 = n1 + n2; // n3 valdrá la suma de n1 y n2. Es decir: 7
```

Otra opción es declarar y asignar la variable en una única instrucción.

```
double n1 = 5.5;
string nombre = "Pedro";
```

Las variables del mismo tipo las podemos agrupar en una única declaración.

```
double n1 = 5.5, n2 = 4.2;
```

Constantes

Una constante es similar a una variable, pero solo se le puede asignar valor una vez. Una vez asignado el valor ya no se le podrá cambiar (error). Los nombres de las constantes suelen escribirse en mayúsculas. En el caso de estar compuestas por varias palabras, se utiliza el carácter de subrayado '_' para separarlas.

```
const int MAX_ITEMS = 12;
MAX_ITEMS = 14; // ¡ERROR!
```

Operaciones numéricas

Con valores numéricos, ya sea con el valor directamente o esté almacenado en una variable podemos realizar operaciones aritméticas como la suma, resta, multiplicación, división y resto. También se pueden hacer operaciones con varios operandos y utilizar paréntesis para agrupar operaciones.

```
double n1 = 5.5, n2 = 4.2;
double suma = n1 + n2;
double resta = n1 - n2;
double multi = n1 * n2;
double division = n1 / n2;
int resto = 5 % 2; // resto vale 1
```

Si queremos actualizar el valor de una variable a partir de su valor actual. Por ejemplo sumarle 2, o dividirla entre 2. Podemos utilizar los operadores +=, -=, *=, /=, que realizan la operación con la variable y luego le reasignan el valor actualizado:

```
int total = 0;
total = total + 4;
total += 4; // Equivale a lo anterior
total *= 2; // Equivale a total = total * 2
Console.WriteLine(total); // Imprime 16 -> (0 + 4 + 4) * 2
```

Además, tenemos las operaciones **incremento** y **decremento** que aumentan o disminuyen el valor de una variable (número entero) en una unidad.

```
int n = 0;
n++; // Equivale a hacer n = n + 1 o n += 1
Console.WriteLine(n); // Imprime 1
n--; // Equivale a hacer n = n - 1 o n -= 1
Console.WriteLine(n); // Imprime 0
```

En realidad hay 2 tipos de incremento y decremento con sutiles diferencias. **Preincremento** (++variable) → incrementa el valor y luego se sustituye por el valor actualizado. **Postincremento** (variable++) → primero se sustituye por el valor actual y al terminar la instrucción se incrementa. Lo mismo para decrementar.

```
int num = 0;
Console.WriteLine(num++); // Imprime 0 y luego incrementa
Console.WriteLine(++num); // Primero incrementa. Imprime 2
```

Operaciones con caracteres

Los caracteres se almacenan internamente como un entero de 16 bits (equivale a un tipo short), por lo que podemos hacer operaciones como sumar o restar por ejemplo. El número que se almacena es la posición en la tabla de caracteres [Unicode](#).

Aquí tenemos un ejemplo de cómo averiguar la posición de una letra en el alfabeto inglés (le restamos la letra 'a' para saber la diferencia y le sumamos 1).

```
Console.WriteLine('m' - 'a' + 1); // Imprime 13
```

Para obtener su valor numérico, bastaría con asignar el carácter a una variable de tipo entero, ya que internamente hemos dicho que se almacena un valor numérico. Lo que cambia según el tipo de la variable es la forma de imprimirlo.

```
int c = 'a';
Console.WriteLine(c); // Imprime 97
```

Concatenación de cadenas

Si queremos concatenar 2 o más textos en uno solo, podemos utilizar varias estrategias. La más directa sería unirlos con el símbolo +.

```
Console.WriteLine("Hola " + "mundo!"); // Imprime "Hola mundo!"
```

Algunas de las partes del texto a concatenar suelen estar almacenadas en variables. Si alguna variable no es del tipo string, será transformada a cadena automáticamente antes de concatenarla al resto del texto.

```
string nombre = "Juan";  
int edad = 25;  
Console.WriteLine(nombre + " tiene " + edad + " años"); //Juan tiene 25 años
```

¡Cuidado!: Si intentamos concatenar 2 variables que contienen valores numéricos, estos se sumarán, ya que se usa el mismo símbolo que para la operación de sumar números. La solución sería transformar al menos una de ellas a string.

```
int prefijo = 34;  
int telefono = 923423243;  
Console.WriteLine(prefijo + telefono); // 923423277 (suma)  
Console.WriteLine(prefijo.ToString() + telefono); // 34923423243 (concatena)
```

Otro método para concatenar variables en un texto es usar la sustitución de variables. Para ello debemos encerrar todo dentro de comillas dobles y en los lugares donde queremos sustituir variables, pondremos llaves con un número empezando por cero que iremos incrementando por cada variable que queramos añadir. Finalmente se pondrá la lista de variables separadas por coma.

```
Console.WriteLine("{0} tiene {1} años", nombre, edad);
```

Si en lugar de imprimir por consola, queremos asignar la cadena completa a otra variable con este método, debemos usar String.Format. En realidad, siempre sería necesario con este tipo de sustitución de variables, pero al imprimir por consola es una operación implícita.

```
string frase = String.Format("{0} tiene {1} años", nombre, edad);  
Console.WriteLine(frase); //Juan tiene 25 años
```

Desde la versión 6 de C# existe otra manera de sustituir variables en cadenas más directa. Si comenzamos la cadena con \$ antes de las comillas dobles, bastaría con poner el nombre de la variable entre llaves para que se sustituya por su valor.

```
Console.WriteLine($"{nombre} tiene {edad} años");
```

Operaciones lógicas y de comparación

Como ya hemos visto, el tipo de datos lógico (bool) almacena un valor **true** (verdadero) o **false** (falso).

Lo primero que debemos saber es que hay operaciones, como las de comparación que devuelven este tipo de valores. Los operadores de comparación son: '<' (menor), '>' (mayor), '<=' (menor o igual), '>=' (mayor o igual), '==' (iguales), '!=' (distintos). Ejemplos:

```
Console.WriteLine(5 < 6); // True  
Console.WriteLine(7 >= 10); // False  
  
int a = 12;  
int b = 6;
```



```
Console.WriteLine(a == b); // False
Console.WriteLine(a != b); // True
```

También podemos comparar si dos cadenas son iguales con estos operadores, aunque se suele preferir utilizar Equals.

```
string cadena = "abcd";
bool compruebaCadena = cadena == "abcd"; // True
compruebaCadena = cadena.Equals("abcd"); // True
```

Tenemos también 3 operadores lógicos. La negación '!' que niega el valor lógico a continuación (false → true, true → false), el operador **and** '&&' y el operador **or** '||'. Las reglas básicas son:

- Con el operador and, ambas condiciones deben ser verdaderas para que el resultado global sea cierto.
- Con el operador or, basta con que una de las 2 condiciones sea cierta.

P	Q	P && Q (and)	P Q (or)	!P (not)
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

```
int edad = 25;
int anyosCarnet = 6;
string sexo = "Masculino";
string nombre = "Pepe";

bool esPepeMayor = edad >= 18 && nombre == "Pepe";
Console.WriteLine(esPepeMayor); // True
```

```
bool tieneDescuento = (sexo == "Masculino" && anyosCarnet > 10) || (sexo ==
"Femenino" && anyosCarnet > 5);
Console.WriteLine(tieneDescuento); // False
```

Tipos anónimos y dinámicos

En C# también podemos utilizar los tipos anónimos **var** y **dynamic**. Esto se utiliza cuando no queremos definir el tipo de una variable cuando la declaramos.

En el caso de **var**, se dice que el tipo es implícito. Es obligatorio que asignemos el valor al declarar la variable. El tipo de la variable será el del valor asignado.

```
var a = 12;
var b = "Hola";
System.Console.WriteLine(a.GetTypeCode()); // Int32
System.Console.WriteLine(b.GetTypeCode()); // String
```

En el caso de **dynamic**, se comportaría como en un lenguaje débilmente tipado. Es decir, a la variable se le podría asignar diferentes tipos de datos en tiempo de

ejecución. Sólo se deben usar este tipo de variables si está muy justificado, ya que es más complicado detectar errores y el rendimiento se ve ligeramente afectado.

```
dynamic a;
a = 12;
System.Console.WriteLine(a.GetTypeCode()); // Int32
a = "Hola";
System.Console.WriteLine(a.GetTypeCode()); // String
```

Conversión de tipos de datos

Cuando trabajamos con tipos de datos compatibles como números enteros, decimales y caracteres, por ejemplo. Podemos hacer una conversión del dato de un tipo al otro. Para ello, cuando leemos el valor, le cambiamos el tipo poniendo entre paréntesis delante el nuevo tipo.

Por ejemplo, para quitarle los decimales a un número, lo transformamos a entero:

```
double dec = 23.63;
int entero = (int)dec; // Eliminamos la parte decimal
Console.WriteLine(entero); // 23
```

Las operaciones con números enteros siempre dan como resultado otro número entero. Esto implica que al hacer una división de 2 números enteros, no obtendremos la parte decimal. La solución es convertir al menos uno de los 2 operandos en número decimal (double). El resultado será en este caso otro número decimal.

```
int a = 11;
int b = 4;
Console.WriteLine(a / (double)b); // 2,75
```

También se pueden hacer conversiones entre char e int por ejemplo.

```
char c = 'b';
// Imprime -> El caracter 'b' tiene la posición 98
Console.WriteLine($"El caracter unicode '{c}' tiene la posición {(double)c}");

int pos = 103;
// Imprime -> El caracter con la posición 103 es 'g'
Console.WriteLine($"El caracter con la posición {pos} es '{(char)pos}'");
```

También hay funcionalidades integradas en el lenguaje para convertir entre tipos no equivalentes. Por ejemplo, para transformar una cadena a número podemos usar **Int32.Parse** que devuelve un int (Int16 → short, Int64 → long).

```
string numStr = "342";
int num = Int32.Parse(numStr);
Console.WriteLine(numStr + 100); // 342100 (concatena)
Console.WriteLine(num + 100); // 442 (suma)
```

También existe el equivalente para números decimales (Double.Parse):

```
string numStr = "23.43";
double num = Double.Parse(numStr);
```

Y podemos transformar cualquier valor a string concatenándole **.ToString()**.

```
int n1 = 24;
```

```
Console.WriteLine(n1.ToString() + "65"); // 2465
```

Otra opción es usar el formateo de cadenas (String.Format o \$"{}")

```
double n2 = 53.2435;
string cadena2 = String.Format("{0:F2}", n2);
System.Console.WriteLine(cadena2); // 53,24
```

```
double n3 = 23.4325;
string cadena3 = $"{n3:F2}";
Console.WriteLine(cadena3); // 23,43
```

ToString también permite formatear la cadena. En este caso habría que poner directamente el formato sin usar llaves ni dos puntos.

```
double n = 23.4325;
// Todos estos formatos son equivalentes y muestran 23,43
Console.WriteLine(n.ToString("F2")); // F -> float (decimal)
Console.WriteLine(n.ToString("N2")); // N -> Número (equivale a lo anterior)
Console.WriteLine(n.ToString("0.00"));
Console.WriteLine(n.ToString("#.##"));
```

Para obtener el valor entero de un char que contiene un dígito numérico usaríamos **Char.GetNumericValue** de esta manera:

```
char c1 = '4';
char c2 = '9';
Console.WriteLine(c1 + c2); // 109 (suma de sus posiciones en tabla Unicode)
Console.WriteLine(Char.GetNumericValue(c1) + Char.GetNumericValue(c2)); // 13
```

Finalmente, usando Convert se pueden hacer conversiones entre casi cualquier tipo de datos.

```
double dec = 23.4325;
int ent = Convert.ToInt32(dec);
string str = Convert.ToString(dec);
bool b = Convert.ToBoolean("False");
```

Escribir y leer de consola

Ya hemos visto ejemplos de como podemos imprimir texto por la consola para informar al usuario. Para ello usamos **Console.Write** y **Console.WriteLine**. La diferencia entre ambos es que el segundo agrega un salto de línea después de escribir la cadena.

```
Console.WriteLine("Hola");  
Console.WriteLine("Mundo");  
Console.Write("Hola");  
Console.Write("Mundo");
```

```
Hola  
Mundo  
HolaMundo
```

Caracteres de salto de línea y tabulado

Si queremos generar una cadena con varias líneas de texto podemos utilizar el carácter especial ‘\n’. Cada vez que aparece este carácter se genera un salto de línea y el texto que haya a continuación aparecerá en la línea siguiente:

```
Console.WriteLine("Hola\nMundo");
```

```
Hola  
Mundo
```

Para generar un espacio de tabulador necesitamos el carácter especial ‘\t’.

```
Console.WriteLine("Elige una opción");  
Console.WriteLine("\t1) Añadir producto");  
Console.WriteLine("\t2) Borrar producto");
```

```
Elige una opción  
    1) Añadir producto  
    2) Borrar producto
```

Equivalente a lo anterior sería:

```
Console.WriteLine("Elige una opción\n\t1) Añadir producto\n\t2) Borrar  
producto");
```

Si situamos una arroba “@” delante de la cadena (literal string), esta dejar de interpretar los caracteres especiales. Los saltos de línea y tabulados los pondríamos tal cual al escribir la cadena (admite cadenas de varias líneas). Sin embargo, esto podría generar confusión con la indentación del código como se puede observar:

```
namespace Mi_Proyecto  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            Console.WriteLine(@"Elige una opción  
1) Añadir producto  
2) Borrar producto");  
        }  
    }  
}
```

```
}
```

Formatear la salida de texto

Utilizando `String.Format` (para guardar el resultado en una variable) o directamente con `Console.WriteLine` (para escribir por consola), tenemos una serie de herramientas para formatear diferentes tipos de texto y controlar cosas como el número de decimales a mostrar, el espaciado de un texto a derecha e izquierda, mostrar un número con ceros a la izquierda, etc.

Utilizando lo que hemos aprendido hasta ahora, es complicado alinear correctamente el texto cuando queremos mostrarlo en columnas:

```
string nombre1 = "Paco";
string nombre2 = "Anastasia";
double salario1 = 20300.24;
double salario2 = 24439.6;

Console.WriteLine("          NOMBRE          SALARIO");
Console.WriteLine("-----");
Console.WriteLine("{0}          {1}", nombre1, salario1);
Console.WriteLine("{0}          {1}", nombre2, salario2);
```

```
          NOMBRE          SALARIO
-----
Paco          20300,24
Anastasia     24439,6
```

Recordemos que poniendo `$` delante de la cadena, podíamos insertar directamente las variables entre las llaves en lugar del orden de las mismas.

```
Console.WriteLine("          NOMBRE          SALARIO");
Console.WriteLine("-----");
Console.WriteLine($"          {{nombre1}}          {{salario1}}");
Console.WriteLine($"          {{nombre2}}          {{salario2}}");
```

Ahora vamos a ver como indicar un número de espacios fijo que debe ocupar un valor mostrado, y como mostrar un número con una cantidad determinada de decimales.

Entre las llaves, justo después del valor a mostrar, podemos añadir un valor numérico separado por una coma → **{variable, 12}**. Este valor indica la cantidad de espacios que va a ocupar como mínimo dicho valor. Si ocupa menos, se rellenará con espacios en blanco a la izquierda.

```
Console.WriteLine($"{{"NOMBRE",12}}{{"SALARIO",14}}");
Console.WriteLine("-----");
Console.WriteLine($"{{nombre1,12}}{{salario1,14}}€");
Console.WriteLine($"{{nombre2,12}}{{salario2,14}}€");
```

```
          NOMBRE          SALARIO
-----
Paco          20300,24€
Anastasia     24439,6€
```

Si queremos alinear el texto a la izquierda (espacios en blanco a la derecha), utilizaremos un número negativo en lugar de positivo.

```
Console.WriteLine($"{{"NOMBRE",-12}}{{"SALARIO",-14}}");
```

```

Console.WriteLine("-----");
Console.WriteLine($"{nombre1,-12}{salario1+"€",-14}");
Console.WriteLine($"{nombre2,-12}{salario2+"€",-14}");

```

NOMBRE	SALARIO
Paco	20300,24€
Anastasia	24439,6€

Esto está mejor, aunque todavía podemos dejarlo mejor si mostramos un número de decimales fijo. Para ello indicamos con 'F' seguido del número de decimales → **{precio:F2}**. La letra 'F' es para indicar que se trata de un número decimal o de coma flotante (float en inglés).

```

Console.WriteLine($"{ "NOMBRE",12}{ "SALARIO",14}");
Console.WriteLine("-----");
Console.WriteLine($"{nombre1,12}{salario1,14:F2}€");
Console.WriteLine($"{nombre2,12}{salario2,14:F2}€");

```

NOMBRE	SALARIO
Paco	20300,24€
Anastasia	24439,60€

Como curiosidad indicar que además del tipo F, hay muchos otros, entre ellos el de moneda → 'C' (Coin). También le indicamos los decimales a mostrar. Te inserta automáticamente el símbolo de la moneda y te formatea el número mejor.

```

Console.WriteLine($"{ "NOMBRE",12}{ "SALARIO",14}");
Console.WriteLine("-----");
Console.WriteLine($"{nombre1,12}{salario1,14:C2}");
Console.WriteLine($"{nombre2,12}{salario2,14:C2}");

```

NOMBRE	SALARIO
Paco	20.300,24 €
Anastasia	24.439,60 €

Si queremos mostrar un número entero con ceros a la izquierda, indicaremos que se trata de un número entero con 'D' seguido del número de dígitos que debe ocupar dicho número. Si ocupa menos, se completa con ceros a la izquierda.

```

int dia = 4;
int mes = 6;
int anyo = 2013;
Console.WriteLine($"Fecha: {dia:D2}/{mes:D2}/{anyo:D4}");

```

Fecha: 04/06/2013

Hay muchas posibilidades más que no hemos visto aquí para formatear fechas, números, etc. Algunas las veremos más adelante, conforme vayamos avanzando. En cualquier caso siempre se pueden ver ejemplos en la [documentación oficial](#).

Leer de la consola

Para leer de la consola, generalmente usamos **Console.ReadLine**. Esto nos devuelve lo que el usuario ha escrito en consola hasta que ha pulsado la tecla enter. Si queremos leer un solo carácter podríamos usar **Console.Read** (nos devuelve el código numérico del carácter escrito), o **Console.ReadKey** (nos devuelve información sobre la tecla pulsada).

```
Console.WriteLine("Dime tu nombre: ");
string nombre = Console.ReadLine();
System.Console.WriteLine($"Hola {nombre}");

Dime tu nombre:
Arturo ← Escrito por el usuario
Hola Arturo

Console.WriteLine("Presiona una tecla");
var tecla = Console.ReadKey(true); // No muestra la tecla pulsada

System.Console.WriteLine($"Tecla pulsada: {tecla.KeyChar}");

Presiona una tecla
Tecla pulsada: d
```